

HTML SVG

A Practical, From-Zero Learning Guide

Part 2 — SVG Paths

Paths are the foundation of advanced SVG drawing. Once you understand the `<path>` element, you can draw shapes that cannot be described easily with rectangles, circles, lines and polygons.

Goal of this part: Learn how the `d` attribute works, understand absolute and relative commands, draw straight segments, curves and arcs, and read path data written by SVG editors.

1. What is a Path?

A path is a sequence of drawing instructions. The instructions are stored in the **d** attribute.

```
<svg viewBox="0 0 300 200">
  <path d="M 30 150 L 270 50"
        stroke="black"
        stroke-width="6"
        fill="none" />
</svg>
```

The path above says: **move to (30,150)**, then **draw a line to (270,50)**.

2. The d Attribute

The **d** attribute contains commands and numbers. Commands tell SVG what type of drawing operation to perform; numbers provide coordinates or other parameters.

```
d="M 20 20 L 100 80 L 180 20 Z"
```

This can be read as: Move → Line → Line → Close.

3. The Move Command — M

Uppercase **M** uses absolute coordinates.

```
<path d="M 50 100"
      stroke="black"
      stroke-width="5"
      fill="none" />
```

Moving does **not** draw a line. It simply places the invisible path cursor at the specified location.

4. The Line Command — L

```
<path d="M 40 140 L 260 40"
      stroke="black"
      stroke-width="5"
      fill="none" />
```

L x y draws a straight line from the current point to **(x,y)**.

5. Multiple L Commands

```
<path d="M 40 140
        L 100 40
        L 160 140
        L 220 40"
      stroke="black"
      stroke-width="5"
      fill="none" />
```

A path can contain many commands. Each new command continues from the previous endpoint.

6. Closing a Path — Z

```
<path d="M 50 150
        L 150 30
        L 250 150
        Z"
      fill="gold"
      stroke="black"
      stroke-width="4" />
```

Z closes the current subpath by connecting the current point back to the starting point.

Closing with **Z** is usually better than manually drawing a final line when you want a properly closed shape.

7. The Horizontal Line — H

```
<path d="M 40 100 H 260"
      stroke="black"
      stroke-width="5"
      fill="none" />
```

H x changes only the x-coordinate. The y-coordinate remains the same.

8. The Vertical Line — V

```
<path d="M 150 30 V 170"
      stroke="black"
      stroke-width="5"
      fill="none" />
```

V y changes only the y-coordinate. The x-coordinate remains the same.

9. Absolute vs Relative Commands

This is one of the most important path concepts. Uppercase commands use absolute coordinates. Lowercase commands use coordinates relative to the current point.

```
<!-- Absolute -->
<path d="M 50 50 L 150 50 L 150 120" />

<!-- Relative -->
<path d="m 50 50 l 100 0 l 0 70" />
```

Both paths produce the same geometry. In the relative version, **l 100 0** means move 100 units right from the current point.

Remember:

- **M / L / H / V / C / S / Q / T / A** → absolute
- **m / l / h / v / c / s / q / t / a** → relative

10. A Simple House with Paths

```
<svg viewBox="0 0 400 300">
  <path d="M 70 140 L 200 40 L 330 140 Z"
        fill="#f59e0b" />

  <path d="M 90 135 V 260 H 310 V 135"
        fill="#60a5fa"
        stroke="#111"
        stroke-width="4" />

  <path d="M 175 260 V 190 H 225 V 260 Z"
        fill="#92400e" />
</svg>
```

This example demonstrates that complicated illustrations can be assembled from simple path instructions.

11. Horizontal and Vertical Commands Together

```
<path d="M 40 40
        H 260
        V 160
        H 40
        Z"
      fill="lightblue"
```

```
stroke="black"
stroke-width="4" />
```

This creates a rectangle-like shape entirely with a path. The same shape could normally be created more simply with `<rect>`.

12. Cubic Bézier Curves — C

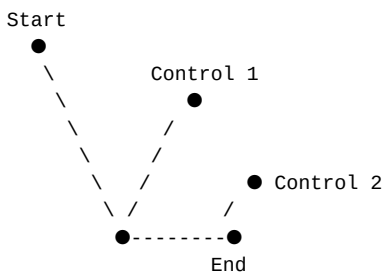
The cubic Bézier command creates a curve using **two control points** and an endpoint.

```
<path d="M 30 150
      C 80 20, 220 20, 270 150"
      fill="none"
      stroke="black"
      stroke-width="6" />
```

Syntax: **C** *x1 y1, x2 y2, x y*. The first pair is control point 1, the second pair is control point 2, and the final pair is the endpoint.

Think of control points as invisible handles that pull the curve.

13. Understanding Cubic Control Points



The curve does not necessarily pass through the control points. Instead, the control points influence the direction and amount of bending.

14. Smooth Cubic Curves — S

```
<path d="M 30 130
      C 70 20, 130 20, 170 130
      S 270 240, 330 130"
      fill="none"
      stroke="black"
      stroke-width="5" />
```

The **S** command creates a smooth continuation from the previous cubic curve. SVG reflects the previous control point around the current point to determine the first control point of the smooth segment.

15. Quadratic Bézier Curves — Q

A quadratic curve uses **one control point**.

```
<path d="M 30 150
      Q 150 10, 270 150"
      fill="none"
      stroke="black"
      stroke-width="6" />
```

Syntax: **Q** *cx cy, x y*. The first pair is the control point and the second pair is the endpoint.

16. Smooth Quadratic Curves — T

```
<path d="M 20 120
      Q 70 20, 120 120
```

```

    T 220 120
    T 320 120"
    fill="none"
    stroke="black"
    stroke-width="5" />

```

The **T** command creates a smooth continuation of a quadratic curve.

17. Cubic vs Quadratic

Command	Control points	Main idea
C	2	Flexible cubic Bézier curve
S	1 explicit + reflected previous	Smooth cubic continuation
Q	1	Quadratic Bézier curve
T	Reflected previous	Smooth quadratic continuation

18. Elliptical Arcs — A

Arcs allow a path to draw part of an ellipse or circle. The syntax looks more complicated because SVG needs enough information to choose which arc to draw.

```

<path d="M 50 150
    A 100 100 0 0 1 250 150"
    fill="none"
    stroke="black"
    stroke-width="6" />

```

The basic syntax is:

```
A rx ry rotation large-arc-flag sweep-flag x y
```

Meaning:

- **rx** — horizontal radius
- **ry** — vertical radius
- **rotation** — rotation of the ellipse
- **large-arc-flag** — 0 for smaller arc, 1 for larger arc
- **sweep-flag** — controls the direction of the arc
- **x y** — endpoint

19. Why the Arc Command Looks Difficult

An ellipse can connect two points in several possible ways. SVG therefore needs flags to tell the renderer which of the possible arcs you want.

Do not try to memorize the arc command all at once. First learn the idea: **radius** → **rotation** → **choose large/small arc** → **choose direction** → **endpoint**.

20. Making a Circle with Four Arcs

```

<path d="
    M 200 50
    A 150 150 0 0 1 200 350
    A 150 150 0 0 1 200 50
"

```

```
fill="none"
stroke="black"
stroke-width="6" />
```

For real work, use **<circle>** when you simply need a circle. Use arcs when you need only part of a circular or elliptical shape.

21. Combining Commands

The real power of paths appears when commands are combined.

```
<path d="
  M 40 150
  L 80 80
  Q 150 10 220 80
  C 250 110 280 120 330 80
  L 360 150
  Z
"
fill="#fbbf24"
stroke="#111827"
stroke-width="4" />
```

A single path can contain straight lines, curves, arcs and multiple subpaths.

22. Multiple Subpaths

A new **M** command can start another subpath without connecting it to the previous one.

```
<path d="
  M 30 30 L 100 30 L 65 90 Z
  M 150 30 L 220 30 L 185 90 Z
"
fill="skyblue"
stroke="black"
stroke-width="3" />
```

This lets one **<path>** contain several separate shapes.

23. Fill Rules

When a path crosses itself or contains holes, SVG needs a rule to determine which regions should be filled.

```
<path d="
  M 40 40 H 260 V 260 H 40 Z
  M 90 90 H 210 V 210 H 90 Z
"
fill="tomato"
fill-rule="evenodd" />
```

fill-rule commonly uses **nonzero** or **evenodd**. The even-odd rule is particularly useful when you want nested regions to behave like holes.

24. Path Data Can Be Compact

Commas and spaces are often optional where SVG can clearly separate numbers. This is why path data generated by design tools can look strange at first.

```
M10 10L90 10L90 90Z
```

This is valid and means the same basic sequence as a more spaced-out version.

25. Path Commands Cheat Sheet

Command	Meaning
M / m	Move to
L / l	Line to
H / h	Horizontal line
V / v	Vertical line
Z / z	Close path
C / c	Cubic Bézier
S / s	Smooth cubic Bézier
Q / q	Quadratic Bézier
T / t	Smooth quadratic Bézier
A / a	Elliptical arc

26. Mini Project — A Simple Cloud

```
<svg viewBox="0 0 500 300">
  <path d="
    M 130 210
    C 85 210 65 175 75 140
    C 85 105 120 90 150 105
    C 165 60 215 45 250 75
    C 280 45 335 65 345 110
    C 390 105 425 135 425 175
    C 425 200 405 210 380 210
    Z
  "
  fill="#93c5fd"
  stroke="#1e3a8a"
  stroke-width="5"/>
</svg>
```

Study the command sequence and try moving individual control points. Your goal is to understand why changing a control point changes the curve.

27. Practice Projects

- Draw a heart using only a path.
- Draw a crescent moon using two arcs.
- Draw a speech bubble using lines and curves.
- Draw a simple wave using C and S.
- Draw a leaf using two Bézier curves.
- Draw a mountain landscape using lines and curves.
- Recreate a simple icon from a reference image using paths.
- Convert one of your Part 1 shapes into a path and compare the two.

28. Important Learning Advice

Do not memorize every coordinate or every arc flag. Learn to **read the grammar** of a path. When you see **M**, identify the starting point; when you see **L**, identify the next endpoint; when you see **C**, identify the two handles and endpoint.

For serious SVG work, you will eventually use vector editors and browser developer tools, but understanding path syntax lets you edit and generate SVG confidently instead of treating the path data as mysterious text.

29. What's Next?

Part 3 will cover **groups and transformations**. You will learn how to move, rotate, scale and skew SVG elements; how nested coordinate systems work; and how to build reusable graphics instead of manually repeating shapes.

Next major concepts: <g> → translate → rotate → scale → skew → transform-origin → nested transforms → reusable SVG structures.